

2.5 - Comunicação e propriedade de dados

Como serviços conversam sem se acoplar

Padrões de comunicação entre serviços

Síncrono e assíncrono

- **Síncrona via REST ou gRPC para resposta imediata**
 - Simples de entender, mas cria acoplamento temporal
- **Assíncrona via mensageria para desacoplar no tempo**
 - Produtor e consumidor não precisam estar online juntos
- **Cada estilo tem trade-off de latência e erro**
 - *Síncrono propaga falha, assíncrono adia o resultado*
- **Escolha pelo acoplamento que você pode aceitar**
 - A pergunta é quanto de espera o fluxo tolera

Comunicação síncrona vs. assíncrona

Comunicação síncrona

- Resposta imediata na mesma requisição
- Fluxo simples e fácil de depurar
- Falha de um propaga na cadeia toda
- Acoplamento temporal entre os serviços

Comunicação assíncrona

- Desacopla produtor e consumidor no tempo
- Absorve picos e tolera indisponibilidade
- Exige rastreamento e tratamento de erro próprios
- Resultado eventual, não imediato

Contratos e propriedade de dados

- **O contrato é a única fronteira pública do serviço**
 - Mudança de contrato exige versionamento explícito
- **Cada serviço é o único dono dos seus dados**
 - Acesso externo só pela interface, nunca pelo banco
- **Banco compartilhado destrói a independência**
 - *Uma coluna nova vira falha em outro serviço*
- **Contratos estáveis permitem evoluir por dentro**
 - O serviço muda sem quebrar quem o consome

Banco compartilhado é o anti-pattern

- **Dois serviços e um banco é um monólito disfarçado**
- **O acoplamento de dados anula toda a autonomia**

Banco de dados compartilhado entre serviços é o anti-pattern mais destrutivo do estilo. Quem controla o schema controla os dois serviços, e nenhum deles é realmente independente.

- Se precisam dos mesmos dados, talvez sejam um serviço só

30% das leituras falhando: o banco compartilhado

- **Tracking e rotas dividiam a mesma tabela do banco**
 - Uma coluna nova quebrou as leituras do tracking

LUNALOG

O novo serviço de tracking e o serviço de rotas da LunaLog compartilhavam a mesma tabela. Um desenvolvedor do time de rotas adicionou uma coluna para uma feature aparentemente inocente. No dia seguinte, o tracking falhava em 30% das leituras. Os dois times descobriram que tinham construído microserviços com dependência de banco compartilhado, o anti-pattern mais caro do estilo. A partir dali, propriedade exclusiva de dados deixou de ser teoria e virou regra inegociável.



Comunicação e propriedade de dados no lugar. Ainda assim, existe uma classe de problemas que só aparece quando os serviços entram em produção de verdade, e que surpreende até times experientes. A rede falha, requisições somem no meio do caminho, e nada disso existia no monólito. Você está pronto para a complexidade que ninguém coloca no slide de vendas?

A complexidade real dos microsserviços